

```
In [48]: import numpy as np
import os
import zipfile
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics
from sklearn.metrics import classification_report
import seaborn as sns
import zipfile

zip_path = "CH.SC.U4CSE24156_EX-9.6_1(KNN).zip"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall("/content")

print("ZIP file extracted successfully")
```

ZIP file extracted successfully

```
In [23]: print(os.listdir("/content"))

['testDigits', 'trainingDigits']
```

```
In [25]: def img2vector(filename):
    returnVect = np.zeros((1, 1024))

    fr = open(filename)

    for i in range(32):
        lineStr = fr.readline()

        for j in range(32):
            returnVect[0, 32*i+j] = int(lineStr[j])

    return returnVect
```

```
In [27]: training_path = "/content/trainingDigits"

trainingFileList = os.listdir(training_path)

m = len(trainingFileList)

trainingMat = np.zeros((m, 1024))

hwLabels = []

for i in range(m):

    fileNameStr = trainingFileList[i]

    fileStr = fileNameStr.split('.')[0]

    classNumStr = int(fileStr.split('_')[0])

    hwLabels.append(classNumStr)

    trainingMat[i, :] = img2vector(
        os.path.join(training_path, fileNameStr)
    )
```

```
print("Training samples:", m)
print("Training data shape:", trainingMat.shape)
```

Training samples: 1934

Training data shape: (1934, 1024)

```
In [28]: test_path = "/content/testDigits"

testFileList = os.listdir(test_path)

mTest = len(testFileList)

testMat = np.zeros((mTest, 1024))

testLabels = []

for i in range(mTest):

    fileNameStr = testFileList[i]

    fileStr = fileNameStr.split('.')[0]

    classNumStr = int(fileStr.split('_')[0])

    testLabels.append(classNumStr)

    testMat[i, :] = img2vector(
        os.path.join(test_path, fileNameStr)
    )

print("Test samples:", mTest)
print("Test data shape:", testMat.shape)
```

Test samples: 946

Test data shape: (946, 1024)

```
In [36]: from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(
    n_neighbors=3,
    metric='minkowski',
    p=2
)

knn.fit(trainingMat, hwLabels)
```

```
Out[36]: KNeighborsClassifier
KNeighborsClassifier(n_neighbors=3)
```

```
In [38]: y_pred = knn.predict(testMat)

print(y_pred)
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
5 5 3 6 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6
6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6
6 6 6 6 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
8 8 8 8 8 3 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8
8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8
8 8 8 8 8 8 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9]
```

C:\Users\Purnishamahi\anaconda3\Lib\site-packages\joblib\externals\loky\backend\context.py:136: UserWarning: Could not find the number of physical cores for the following reason:

[WinError 2] The system cannot find the file specified

Returning the number of logical cores instead. You can silence this warning by setting LOKY_MAX_CPU_COUNT to the number of cores you want to use.

warnings.warn(

File "C:\Users\Purnishamahi\anaconda3\Lib\site-packages\joblib\externals\loky\backend\context.py", line 257, in _count_physical_cores

cpu_info = subprocess.run(

^^^^^^^^^^^^^^^^^^^^

File "C:\Users\Purnishamahi\anaconda3\Lib\subprocess.py", line 548, in run
with Popen(*popenargs, **kwargs) as process:

^^

File "C:\Users\Purnishamahi\anaconda3\Lib\subprocess.py", line 1026, in __init_

— self._execute_child(args, executable, preexec_fn, close_fds,

File "C:\Users\Purnishamahi\anaconda3\Lib\subprocess.py", line 1538, in _execute_child

hp, ht, pid, tid = _winapi.CreateProcess(executable, args,

^^

In [40]: **from** sklearn.metrics **import** accuracy_score

accuracy = accuracy_score(testLabels, y_pred)

print("Accuracy:", accuracy)

Accuracy: 0.9873150105708245

In [42]: **from** sklearn.metrics **import** confusion_matrix

conf_mat = confusion_matrix(testLabels, y_pred)

```
print(conf_mat)
```

```
[[ 87  0  0  0  0  0  0  0  0  0]
 [  0 96  0  0  0  0  0  1  0  0]
 [  0  0 92  0  0  0  0  0  0  0]
 [  0  0  0 84  0  0  0  0  0  1]
 [  0  0  0  0 114  0  0  0  0  0]
 [  0  0  0  1  0 106  1  0  0  0]
 [  0  0  0  0  0  0 87  0  0  0]
 [  0  0  0  0  0  0  0 96  0  0]
 [  0  3  0  1  0  0  1  0 86  0]
 [  0  1  0  0  0  1  0  1  0 86]]
```

```
In [44]: from sklearn.metrics import classification_report
```

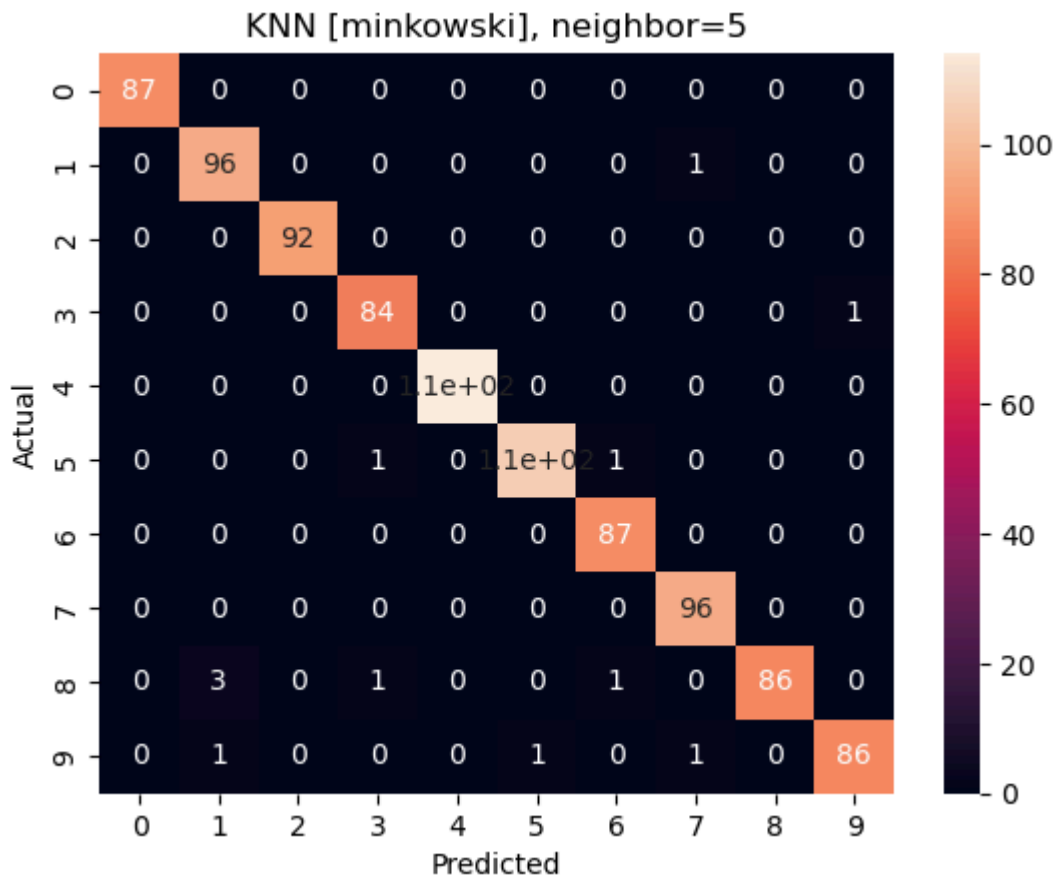
```
print(classification_report(testLabels, y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	87
1	0.96	0.99	0.97	97
2	1.00	1.00	1.00	92
3	0.98	0.99	0.98	85
4	1.00	1.00	1.00	114
5	0.99	0.98	0.99	108
6	0.98	1.00	0.99	87
7	0.98	1.00	0.99	96
8	1.00	0.95	0.97	91
9	0.99	0.97	0.98	89
accuracy			0.99	946
macro avg	0.99	0.99	0.99	946
weighted avg	0.99	0.99	0.99	946

```
In [50]: conf_mat = pd.crosstab(
    testLabels,
    y_pred,
    rownames=['Actual'],
    colnames=['Predicted']
)

sns.heatmap(
    conf_mat,
    annot=True
).set(
    title='KNN [minkowski], neighbor=5'
)
```

```
Out[50]: [Text(0.5, 1.0, 'KNN [minkowski], neighbor=5')]
```



```
In [54]: results = []

for i in [1, 2, 3, 4, 5]:

    model = KNeighborsClassifier(
        n_neighbors=i,
        metric='minkowski',
        p=2
    )

    model.fit(trainingMat, hwLabels)

    y_pred = model.predict(testMat)

    Accuracy_score = metrics.accuracy_score(
        testLabels,
        y_pred
    )

    results.append(Accuracy_score)

print(results)
```

```
[0.9862579281183932, 0.9767441860465116, 0.9873150105708245, 0.9830866807610994,
0.9799154334038055]
```

```
In [56]: models = pd.DataFrame({
    'n_neighbors': ['1', '2', '3', '4', '5'],
    'Accuracy Score': [
        results[0],
        results[1],
        results[2],
        results[3],
```

```
        results[4]  
    ]  
}))  
  
models.sort_values(  
    by='Accuracy Score'  
)  
  
print(  
    models.to_string(index=False)  
)
```

n_neighbors	Accuracy Score
1	0.986258
2	0.976744
3	0.987315
4	0.983087
5	0.979915

In []: